

The Blank Operating System Layer

Team Vertex (Group 23)

Shirley Fang (y266fang)

Pouria Amini (pamini)

Alan Zhu (c63zhu)

Evan He (e7he)

Tony Li (t392li)

Table of Contents

Table of Contents	2
1. Introduction	4
1.1. System Abstract	4
1.2. Glossary of Terms	5
1.3. Notational Conventions	6
2. System Model and Scope	7
2.1. System Domain Model	7
2.2. System Actors and Use Cases	8
2.2.1 Actors	8
2.2.2 Use Cases	8
2.2.3 Use Case Mapping	8
2.2.4 Use Case Diagram	8
2.3. System Assumptions, Variations, and Exceptions	9
2.3.1 Domain Assumptions	9
2.3.2 Variations	9
2.3.3 Exceptions	9
3. User Scenarios	10
3.1 UC1: Submit Task via text or speech.	10
3.1.1. Typical Scenario: Submitting a Task via Text Entry	10
3.1.2. Typical Scenario: Submitting a Task via Speech-to-Text	10
3.1.3. Exception Scenario: Submitting a Task via Faulty microphone.	11
3.1.4. Exceptional Scenario: Non-Supported Language or Gibberish	11
3.1.5. Exceptional Scenario: User Makes a Typo	12
3.1.6. User Interface: The Main Terminal	12
3.2 UC2: Review the message information in the conversation dialog.	13
3.2.1. Typical Scenario: Review a Single-Step Task	13
3.2.2 Alternative Scenario: Review a multi-step task.	14
3.2.3 Exception Scenario: Review an ambiguous task.	14
3.2.4. Exceptional Scenario: User Cancels a Malicious-Appearing Task	14
3.2.5. User Interface: The Task Confirmation Dialog	15
3.2.6. User Interface: The Clarification Dialog	16
3.3 UC3: Approve or disapprove the task action in the conversation dialog.	17
3.3.1. Typical Scenario: Approving a Single-Step Task	17
3.3.2. Exception Scenario: Task Exceeds LLM Token Limits	18
3.3.3 Alternative Exceptional Scenario: Task Involves Non-Supported File Type	19
3.4 UC4: Provide additional context in the conversation dialog.	19
3.4.1. Typical Scenario: Providing Missing Context	19
3.4.2. Exceptional Scenario: Providing Irrelevant/Conflicting Context	20
3.4.3. Alternative Scenario: User Abandons Task When Prompted for Context	21
3.4.4. User Interface: Context Request Dialog	22
3.5 UC5: Acknowledge the error in the task action in the conversation dialog.	23
3.5.1. Typical Scenario: Acknowledging a Task Failure	23

3.5.2. Exceptional Scenario: MCP Server Unresponsive	23
3.5.3. Exceptional Scenario: Client Instance Already Running	24
3.5.4 Alternative Scenario: User manually fixes the error and asks the system to try again.	25
3.5.5. User Interface: The Error Dialogs	25
3.6 UC6: Connect third-party applications to the client via MCP API.	26
3.6.1. Typical Scenario: Establishing Connection and Submitting a Task	26
3.6.2. Exceptional Scenario: Application Sends Incorrect Data	27
3.6.3. Alternative Scenario: Third-Party Application Enters Prompt for the User	27

1. Introduction

1.1. System Abstract

The Blank Operating System Layer is a user interface that makes operating system navigation and control more expressive and intuitive. It consists of a large language model, a single client, and multiple servers that communicate through the Model Context Protocol, with each message written in JavaScript Object Notation format.

The large language model can be provided by an existing system such as ChatGPT, Google Gemini, or a self-hosted local model.

Each server is responsible for a specific group of related functions. In the Blank OSL, these servers are categorized into three primary sets. Local file system manipulation servers handle operations such as searching, reading, and writing files including .txt, .pdf, .zip. System process manager servers allow the system to spawn or terminate processes, inspect process attributes such as CPU usage, memory usage, and process identifiers, or schedule a process to spawn at a specific time. Miscellaneous component servers handle common queries, such as retrieving weather reports, searching Wikipedia, and accessing other online services like emails. Unlike the local file system servers and the process manager servers, the miscellaneous components servers have no strict bounds on the functionalities as it is more like additional feature plugins. Every server exposes the purpose of its available functions, along with parameter types and return values, in a format that follows the Model Context Protocol standard. This design restricts the large language model to these specific functions, and any invalid function calls or parameters are rejected by the server.

The client provides a terminal-style interface that allows the user to issue tasks in natural language through typing in the text entry or using the speech-to-text feature. When a task is submitted, the client forwards it to the large language model, which interprets the user's intent and converts it into a single Model Context Protocol message. If the task cannot be performed based on the available servers, the large language model rejects it and returns an error. If the task is ambiguous, such as creating a temporary file, the large language model may ask for additional details, including the file's name, type, and lifetime in a conversation dialog. Once the client receives the message from the large language model, it displays the task details, including the operation to perform, generated parameters, required permissions, and server dependencies, on a dialog, for the user to review. The user can type yes to continue or no to cancel. If the user chooses to continue, the client sends the message to the appropriate server, which executes the task.

Third-party applications with Model Context Protocol library installed, can interact with the client through its exposed application interface. For instance, it may directly send a task to the client as if the user types in the text entry.

1.2. Glossary of Terms

The Blank OSL: The Blank Operating System Layer.

JSON: JavaScript Object Notation.

API: Application Programming Interface.

LLM: Large Language Model.

UI: User interface.

AI: Artificial intelligence.

OS: Operating system, but restricted to only Windows in The Blank OSL for the sake of simplicity.

MCP: Model Context Protocol, an open-source standard for connecting AI applications to external systems.

Natural Language: Human speaking language, but restricted to only English in The Blank OSL for the sake of simplicity.

Task: An action described by natural language as a paragraph, for example, “getting the titles of the top 10 most popular songs in 2025, then save to my home directory as hello.txt”.

Message: A JSON encoded message following the MCP format.

Client: A client that implements MCP client API.

Server: A server that implements MCP server API.

User: Human user who uses The Blank OSL.

Third-party application: Any external software with MCP support that can communicate with the Blank OSL client via its API.

Dialog: A conversation dialog that contains either error message, or MCP messages with details and ask for user’s confirmation.

1.3. Notational Conventions

User Input: Text that is typed by the user into a text entry field, either in the main terminal or a dialog box.

`` or [Icon]: A graphical user interface element that the user can select, typically with a mouse click or by pressing the Enter key when focused.

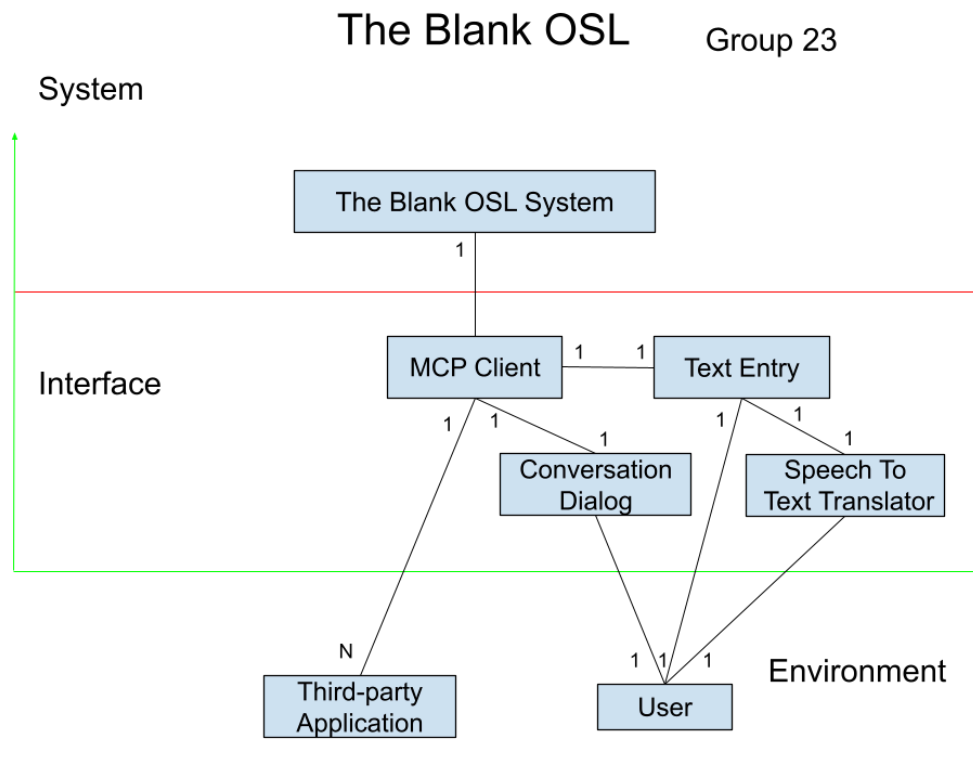
SYSTEM_RESPONSE: Text or output displayed by the Blank OSL system in the main terminal's history area.

> [user_input]: In UI mockups of the main terminal, this format denotes the command prompt and the adjacent text entry area where the user types.

Scenario Tables: All scenarios are presented in a structured, step-by-step table format. This describes the interaction between the actor like the user and the MCP client.

2. System Model and Scope

2.1. System Domain Model



The LLM is in the system domain as the user does not directly interact with it. In fact, it is completely controlled by the MCP library and abstracted away under the MCP client.

The MCP servers are in the system domain as it contains most of the function implementation details. MCP client communicates with multiple MCP servers through the Model Context Protocols.

Third-party applications may directly interact with the MCP client if the third-party applications follow the MCP format.

2.2. System Actors and Use Cases

2.2.1 Actors

User: Interact with the client through text entry, voice to speech translator, and the dialog.

Third-party Applications: Interact with the client through the client API.

2.2.2 Use Cases

UC1: Submit Task via text or speech.

UC2: Review the message information in the conversation dialog.

UC3: Approve or disapprove the task action in the conversation dialog.

UC4: Provide additional context in the conversation dialog.

UC5: Acknowledge the error in the task action in the conversation dialog.

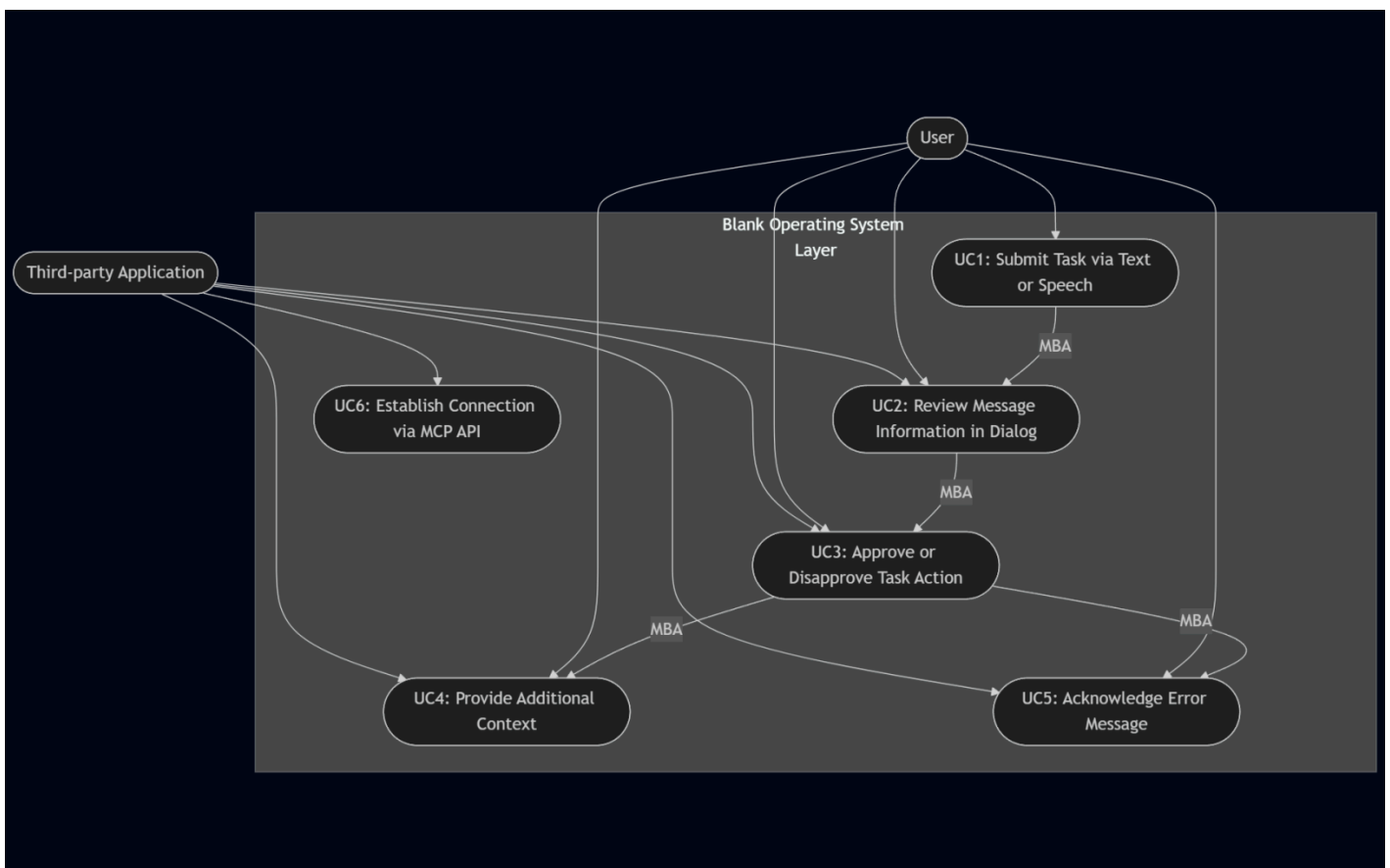
UC6: Connect third-party applications to the client via MCP API.

2.2.3 Use Case Mapping

User through client interface: UC1-UC5

Third-party Application interact programmatically: UC1-6

2.2.4 Use Case Diagram



2.3. System Assumptions, Variations, and Exceptions

2.3.1 Domain Assumptions

1. Only one instance of the Blank OSL client is running at a given time.
2. All Blank OSL components (client, servers, and LLM) function normally.
3. The LLM used has sufficient token capacity for all tasks.
4. The speech-to-text translator performs reliably regardless of the user's accent.
5. The Blank OSL client and servers can finish the task in a reasonable time frame.
6. The user has a computer with basic input hardware such as keyboard and microphone.
7. The user can type, read, and speak English.
8. The task that the user provides matches what the user wants to do.
9. The user understands basic Windows navigation, e.g. the user can navigate the UI.
10. The user and third-party applications do not attempt malicious actions.
11. The third-party application must comply with the Model Context Protocol.

2.3.2 Variations

1. Submit a task via text entry.
2. Submit a task via speech-to-text.
3. Provide additional context through the dialog to complete multi-step tasks.
4. Submit a task by typing terminal commands such as `ls *`, which lists the contents of the current directory.

2.3.3 Exceptions

1. The user forgets the previous instances and opens another one.
2. The MCP servers become temporarily unresponsive due to temporary network disruption.
3. The task length exceeds LLM token limits.
4. The task is described in non-supported natural languages or gibberish.
5. The user's computer is running other high CPU load processes and temporarily slows down the task.
6. The user's microphone is faulty and fails to pick up the user's voice.
7. The user's English vocabulary meaning may differ by the regions and produce different results. For example, "I need a toilet" may mean "need to go to the washroom" in British English, but may mean "want to purchase a toilet" in American English.
8. The user makes a typo in the task.
9. The user is unfamiliar with the Blank OSL's interface design.
10. A third-party program behaves incorrectly and triggers a false positive that appears malicious.
11. The third party application sends incorrect data through the Model Context Protocol.

3. User Scenarios

3.1 UC1: Submit Task via text or speech.

3.1.1. Typical Scenario: Submitting a Task via Text Entry

Step	Actor	System
1.	The user launches the Blank OSL client.	The system displays the Main Terminal interface (Figure 3.1-A). The prompt BLANK_OSL> is active.
2.	The user types a natural language task into the text entry > Find all.txt files in my Documents folder and zip them into 'archive.zip'.	The text appears in the text entry field next to the prompt.
3.	The user presses the Enter key.	The system takes the user's text and forwards it to the LLM for interpretation. The main terminal becomes non-interactive. The LLM interprets the task and generates a valid MCP message. The system displays the "Task Confirmation Dialog".

3.1.2. Typical Scenario: Submitting a Task via Speech-to-Text

Step	Actor	System
1.	The user launches the Blank OSL client.	The system displays the Main Terminal interface (Figure 3.1-A).
2.	The user clicks the [Mic] icon next to the text entry.	The system activates the speech-to-text translator. The [Mic] icon changes to indicate it is "Listening...".
3.	The user speaks the task: "Find all text files	The system transcribes the user's speech in real-time. The transcribed text is "Find all

	in Documents and zip them."	text files in Documents and zip them". appears in the text entry field.
4.	The user finishes speaking and clicks the [Mic] icon again or presses Enter.	The speech-to-text translator deactivates. The system takes the transcribed text and forwards it to the LLM. The system displays the "Task Confirmation Dialog".

3.1.3. Exception Scenario: Submitting a Task via Faulty microphone.

Step	Actor	System
1.	The user launches the Blank OSL client.	The system displays the Main Terminal interface (Figure 3.1-A).
2.	The user clicks the [Mic] icon next to the text entry.	The system activates the speech-to-text translator. The system fails to detect a microphone or fails to get permission to access it. The system displays an inline error message in red text directly below the text entry: Microphone not detected or permission denied. Please check the hardware and OS settings.

3.1.4. Exceptional Scenario: Non-Supported Language or Gibberish

Step	Actor Action (User)	System Response / UI State
1.	The user launches the Blank OSL client.	The system displays the Main Terminal interface (Figure 3.1-A).
2.	The user types a task in a non-supported language or types gibberish. (e.g. "asdfjkl"; or "Où sont mes fichiers?")	The text appears in the text entry field. The system forwards the text to the LLM for interpretation. The LLM is unable to interpret the user's intent or convert it into a valid MCP message. The LLM returns an

		error.
3.	The user presses the Enter key.	The system forwards the text to the LLM for interpretation. The LLM is unable to interpret the user's intent or convert it into a valid MCP message. The LLM returns an error. The system displays the "Task Failure Dialog". The error message in the dialog reads: Task Not Understood: The task could not be interpreted. Please rephrase the request in English.

3.1.5. Exceptional Scenario: User Makes a Typo

Step	Actor Action	System
1.	The user launches the Blank OSL client.	The system displays the Main Terminal interface (Figure 3.1-A).
2.	The user types a task with a minor typo. Fimd all my.txt files in /docs.	The text appears in the text entry field.
3.	The user presses the Enter key.	The system forwards the text Find all my.txt files in /docs. to the LLM. The LLM interprets the user's intent as "Find all my.txt files..." (per Assumption 9, "task... matches what the user wants to do").

3.1.6. User Interface: The Main Terminal

The interface consists of a single window with three main components:

1. A scrollable, read-only area that displays the history of submitted tasks and the final status (e.g., Task cancelled, Task complete).
2. A persistent, single-line prompt at the bottom of the window where the user interacts with the system.
3. A bar at the very bottom of the window displaying the real-time status of system components.

```
Blank OSL (Team Vertex)

SYSTEM_RESPONSE: Welcome to the Blank Operating System Layer.
SYSTEM_RESPONSE: Task 'archive_docs' completed successfully.

> Find all my.txt files...
SYSTEM_RESPONSE: Task cancelled by user.
```

> []

Mic

LLM: Connected MCP Servers: 3/3 Online OS: Windows 11

Figure 3.1-A: The Main Terminal Interface

3.2 UC2: Review the message information in the conversation dialog.

3.2.1. Typical Scenario: Review a Single-Step Task

Step	Actor	System
1.	The user sends the task to the client.	The system displays the "Task Confirmation Dialog" (Figure 3.2-A).
2.	The user reviews it and approves or disapproves.	The system closes the “Task Confirmation Dialog”.

3.2.2 Alternative Scenario: Review a multi-step task.

Step	Actor	System
1.	The user sends the task to the client.	The system displays the "Task Confirmation Dialog" (Figure 3.2-A).
2.	The user reviews it and approves or disapproves.	The system displays another "Task Confirmation Dialog".
	The user keeps reviewing the confirmation dialog.	The system keeps displaying more and more "Task Confirmation Dialog".

3.2.3 Exception Scenario: Review an ambiguous task.

Step	Actor	System
1.	The user sends the task to the client.	The system interprets the user's task as gibberish or ambiguous, and asks for additional context information.
2.	The user provides additional context to the task.	The system displays the "Task Confirmation Dialog".
3.	The user reviews it and approves or disapproves.	The system closes the "Task Confirmation Dialog".

3.2.4. Exceptional Scenario: User Cancels a Malicious-Appearing Task

Step	Actor	System
1.	The user submits a task that could be interpreted as destructive. Delete all the junk files in my temp folder.	The text is submitted to the LLM. The LLM interprets this task and generates an MCP message for a file system delete operation.

		The system displays the "Task Confirmation Dialog". The dialog displays the following interpreted details: Operation: FileSystem.Delete Parameters: Path: 'C:\Users\User\AppData\Local\Temp*' Permissions:
2.	The user, uncertain about the scope of this action, decides to cancel.	The user types no into the dialog's text entry field and presses Enter. The Task Confirmation Dialog closes. The system discards the MCP message. No destructive action is taken. The system displays a status update in the Main Terminal history: SYSTEM_RESPONSE: Task cancelled by user.

3.2.5. User Interface: The Task Confirmation Dialog

This is the most critical UI component in the Blank OSL system. It is a showcase of the "human-in-the-loop" approval step. The contents of this dialog are explicitly defined by the system abstract: "operation to perform, generated parameters, required permissions, and server dependencies". This dialog is modal, meaning it blocks all interaction with the Main Terminal until the user responds (UC3).

Task Confirmation

×

The following task has been generated.
Please review and confirm.

Operation:

FileSystem.ZipFiles

Parameters:

Source: /Documents/*.txt

Destination: /Documents/archive.zip

Required Permissions:

☒ Read: /Documents, Write: /Documents

Server Dependencies:

☒ FileSystem Server

Type 'yes' to continue or 'no' to cancel.

>

[yes|no]

Figure 3.2-A: The Task Confirmation Dialog

3.2.6. User Interface: The Clarification Dialog

This dialog is a second, distinct modal UI pattern. Unlike the yes/no confirmation of Figure 3.2-A, this interface is designed for a conversational exchange to gather more data. It is triggered before the Task Confirmation Dialog, at the point where the LLM identifies ambiguity.


Task Ambiguous
×

I need more information to complete this task.

[LLM-GENERATED QUESTION APPEARS HERE:]

You asked to "create a temporary file." What should the file be named, what type of file is it, and what content should it have?

[USER TYPES RESPONSE HERE]

Type your response here...

Cancel

Submit

Figure 3.3-A: The Clarification Dialog

3.3 UC3: Approve or disapprove the task action in the conversation dialog.

3.3.1. Typical Scenario: Approving a Single-Step Task

Step	Actor	System
1.	The user reviews the information presented in the dialog: Operation, Parameters, Permissions, and Server Dependencies.	The system displays the "Task Confirmation Dialog" (Figure 3.2-A) as a modal window over the Main Terminal. The user confirms the details match to the intent. For example, Operation: FileSystem.Zip, Parameters: Source: /Documents/*.txt, Destination: 'archive.zip'.
2.	The user types yes into the dialog's text	The system validates the "yes" input. The Task

	entry field and presses Enter.	Confirmation Dialog closes. The Main Terminal (Figure 3.1-A) becomes active. The system sends the approved MCP message to the appropriate server (e.g., FileSystem Server). The system displays a status update in the Main Terminal history: SYSTEM_RESPONSE: Task executing... Upon successful completion, the system displays a final status: SYSTEM_RESPONSE: Task 'FileSystem.Zip' completed.
--	--------------------------------	--

3.3.2. Exception Scenario: Task Exceeds LLM Token Limits

Step	Actor	System
1.	The user launches the Blank OSL client.	The system displays the Main Terminal
2.	The user submits an exceptionally large task, for example, by pasting the contents of a 50-page document into the prompt.	The text appears in the text entry field.
3.	The user presses Enter.	The system forwards the large block of text to the LLM for interpretation. The LLM API rejects the request because the input exceeds its maximum token capacity. The LLM returns an error. The system does not display the Confirmation Dialog. Instead, it displays the "Task Failure Dialog". The error message in the dialog reads: Task Failed: Request is too large. The task exceeds the LLM's token limit. Please shorten the request.

3.3.3 Alternative Exceptional Scenario: Task Involves Non-Supported File Type

This scenario describes the system's behavior when the user attempts to submit a task using a file type that the LLM/system integration is not configured to process, violating an implicit or explicit processing constraint.

Step	Actor	System
1.	The user launches the Blank OSL client.	The system displays the Main Terminal
2.	The user attempts to submit a task by clicking a "File Upload" button (or equivalent) and selects a large, complex binary file type, such as a compressed video file (.mov or .mp4), which the system is not designed to parse.	The file name appears temporarily next to the prompt, and the system attempts to process the file input.
3.	The user presses Enter.	The system forwards the file content/reference to the LLM's processing pipeline. The pre-processing or the LLM integration identifies the file type as unsupported for text/data extraction. The system does not display the Confirmation Dialog. Instead, it displays the "Task Failure Dialog". The error message in the dialog reads: Task Failed: Unsupported Input Type. The system cannot process files of type [.mov]. Please use text, PDF, or image formats.

3.4 UC4: Provide additional context in the conversation dialog.

3.4.1. Typical Scenario: Providing Missing Context

Step	Actor	System
------	-------	--------

1.	The user submits a command that is ambiguous or incomplete, such as: "Send the presentation to John."	The system processes the request. The LLM interprets the intent but identifies that the key parameter Source File is missing. The system displays the "Context Request Dialog" (Figure 3.4-A) as a modal window over the Main Terminal.
2.	The user reviews the prompt in the dialog, which clearly states the missing information: "The operation Communication. Send Email requires a Source File. Which file should I send to John?"	The dialog contains a text entry field. The user types the required context, for example: ~/Documents/Q3_Review.pptx and presses Enter.
3.	The user presses Enter.	The system validates the input. The Context Request Dialog closes. The system re-submits the full task (Operation + John + ~/Documents/Q3_Review.pptx) to the LLM for final confirmation generation. The system now displays the standard "Task Confirmation Dialog" with the complete, executable task details: send emails, Parameters: Recipient: John, Source File: ~/Documents/Q3_Review.pptx.
4.	The user reviews the completed task and types yes into the dialog's text entry field and presses Enter.	The system validates the "yes" input. The Task Confirmation Dialog closes, and the task execution begins.

3.4.2. Exceptional Scenario: Providing Irrelevant/Conflicting Context

Step	Actor	System
------	-------	--------

1.	The user is presented with the "Context Request Dialog" from an earlier step (e.g., system needs the Source File for an email). The prompt is: "The operation to send emails requires a Source File. Which file should I send to John?"	The system displays the "Context Request Dialog" (Figure 3.4-A).
2.	The user types irrelevant text, such as: I want pizza now and presses Enter.	The system attempts to re-interpret the full request (Original Intent + New Context). The LLM determines the new context is irrelevant to the required parameter (Source File). The system dismisses the irrelevant context and re-displays the "Context Request Dialog". The prompt is updated to guide the user: "Context Error: The provided text 'I want pizza now' does not appear to be a file path. Please specify the name and location of the file to send."
3.	The user reviews the corrected prompt and types the correct context: ~/Documents/Q3_Review.pptx and presses Enter.	The system validates the input. The Context Request Dialog closes. The system re-submits the full task for final confirmation generation. The system displays the standard "Task Confirmation Dialog" with the complete, executable task details.
4.	The user reviews and approves the task by typing yes and pressing Enter.	The task is approved and execution begins.

3.4.3. Alternative Scenario: User Abandons Task When Prompted for Context

Step	Actor	System
------	-------	--------

1.	The user is presented with the "Context Request Dialog" because the system needs a critical piece of information (e.g., the Source File for an email). The prompt is: "The operation to send emails requires a Source File. Which file should I send to John?"	The system displays the "Context Request Dialog" (Figure 3.4-A).
2.	The user doesn't have the file ready or have changed minds about the task. Instead of providing context, the user types an explicit cancellation command, such as no or cancel, into the dialog's text entry field and presses Enter.	The system validates the input against pre-defined cancellation keywords. The system closes the Context Request Dialog. The Main Terminal becomes active. The system clears the original, incomplete task from memory and displays a status update in the terminal history: SYSTEM_RESPONSE: send emails tasks cancelled by user.
3.	The user continues using the Main Terminal for a new task.	The system is ready to accept a new prompt.

3.4.4. User Interface: Context Request Dialog

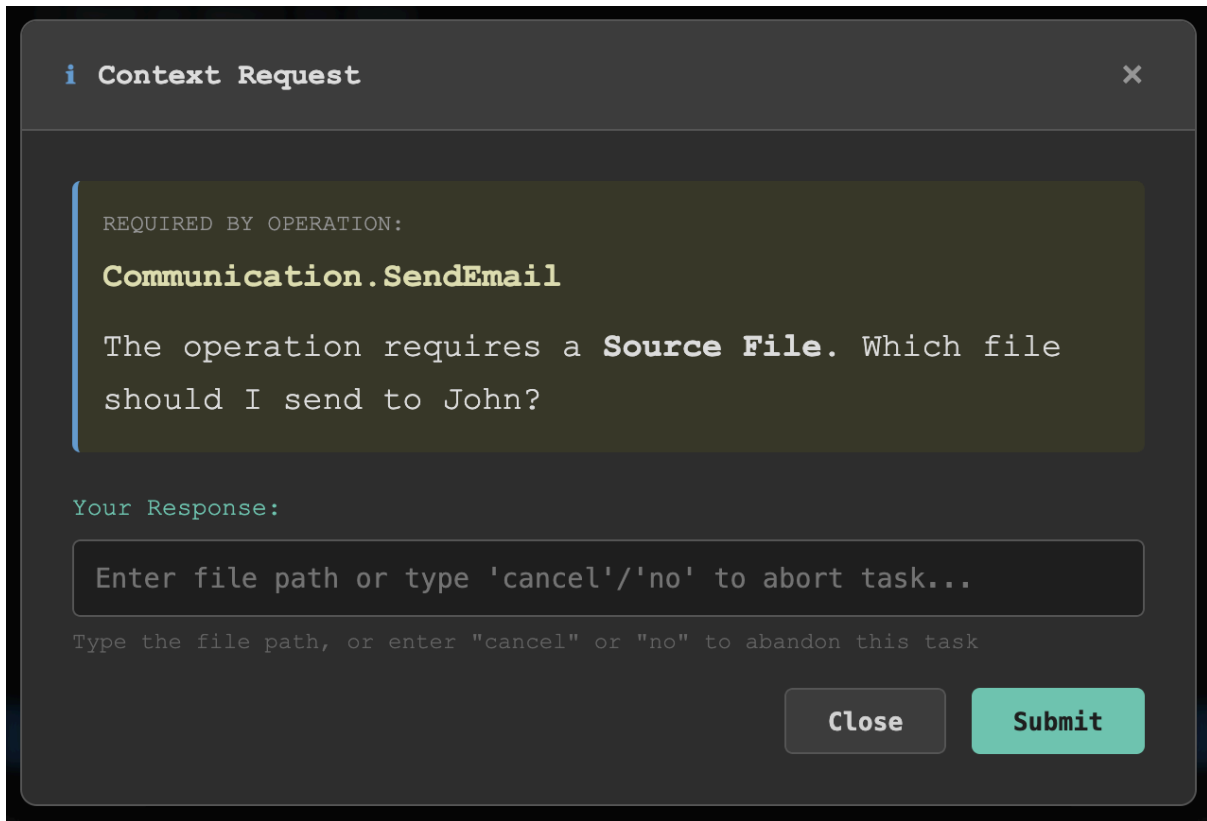


Figure 3.4-A: The Context Request Dialog

3.5 UC5: Acknowledge the error in the task action in the conversation dialog.

3.5.1. Typical Scenario: Acknowledging a Task Failure

Step	Actor	System
1.	The user has approved a task. The system has sent the MCP message to the server.	The MCP Server attempts to execute the task but fails due to a runtime error. The MCP server returns an error message to the client. The system displays the "Task Failure Dialog" (Figure 3.5-B).
2.	The user reads the error message and clicks the [Acknowledge] button.	The Task Failure Dialog closes.

3.5.2. Exceptional Scenario: MCP Server Unresponsive

Step	Actor	System
1.	The user has approved a task. The system has sent the MCP message to the server.	The client waits for a response from the MCP server, but the server is down or the network is disrupted. After a predefined timeout period (e.g., 30 seconds), the client gives up. The system displays the "Task Failure Dialog". The dialog displays the timeout error: Task Failed: The server is unresponsive. Please check the network connection and try again.
2.	The user clicks the [Acknowledge] button.	The Task Failure Dialog closes.

3.5.3. Exceptional Scenario: Client Instance Already Running

Step	Actor Action (User)	System Response / UI State
1.	The user already has an instance of Blank OSL running, but the user forgets and double-clicks the application icon again.	The new instance of the system begins its startup sequence. The system checks to see if another instance is active. It finds one. The system halts the startup of the new instance. The system displays the "Startup Error Dialog" (Figure 3.5-A). The dialog displays the error: Startup Failed: Another instance of Blank OSL is already running.
2.	The user clicks the [OK] button.	The Startup Error Dialog closes. The new application instance terminates. The original, already-running instance is unaffected.

3.5.4 Alternative Scenario: User manually fixes the error and asks the system to try again.

Step	Actor	System
1.	The user has approved a task. The system has sent the MCP message to the server.	The MCP Server attempts to execute the task but fails due to a runtime error. The MCP server returns an error message to the client.
2.	The user sees the error in the output and writes a new task.	The MCP Server attempts to execute the new task.

3.5.5. User Interface: The Error Dialogs

To handle the different types of errors (critical startup failures vs. recoverable task failures), two distinct error dialogs are required.

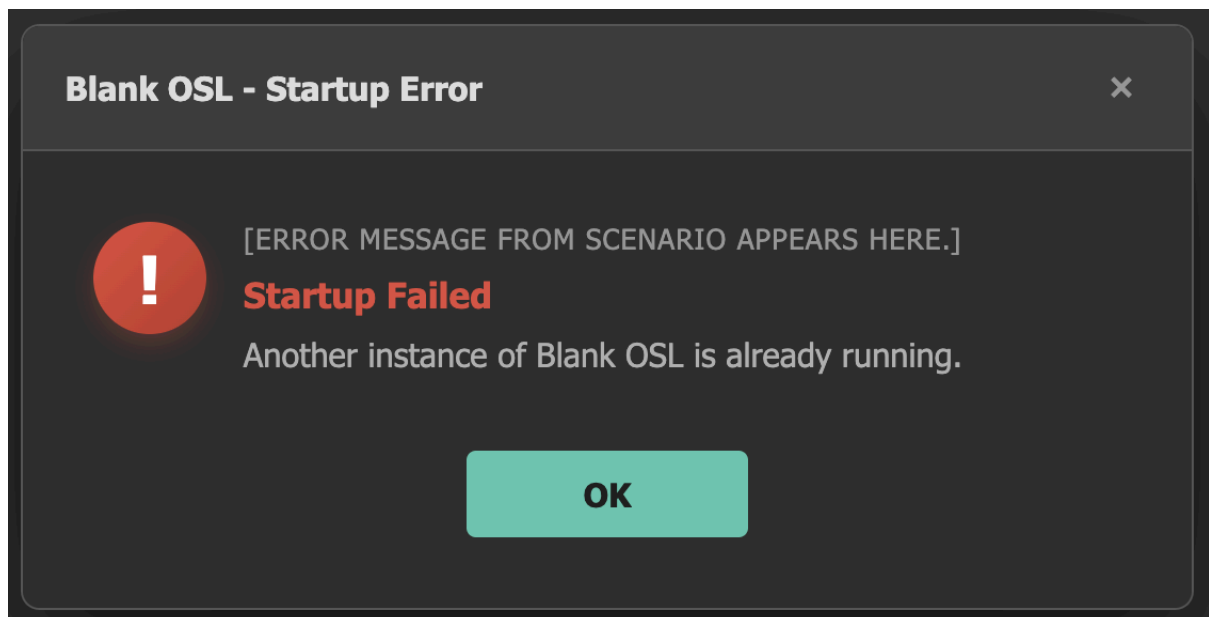


Figure 3.5-A: Startup Error Dialog. This dialog is used for critical, blocking errors that prevent the application from starting. It is a simple, standard OS dialog.

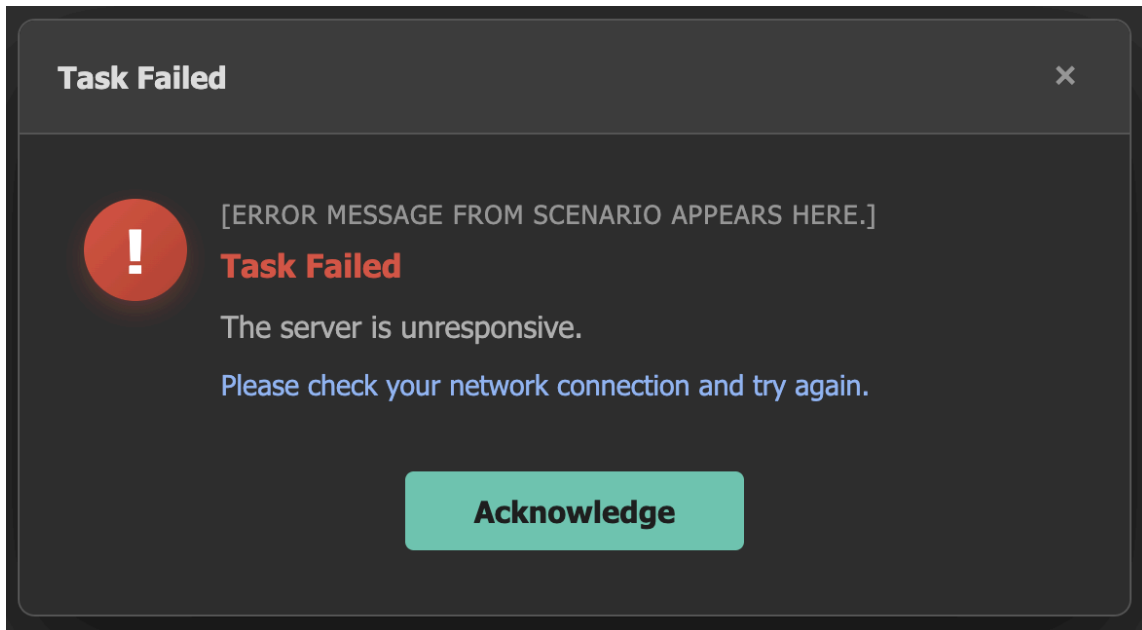


Figure 3.5-B: Task Failure Dialog. This dialog is used for non-critical, recoverable errors that occur after the application is running. It is modal over the Main Terminal.

3.6 UC6: Connect third-party applications to the client via MCP API.

3.6.1. Typical Scenario: Establishing Connection and Submitting a Task

Step	Actor	System
1.	The application establishes a connection to the Blank OSL client's exposed MCP endpoint.	The Blank OSL client acts as an MCP host and accepts the connection.
2.	The application sends a SubmitTask request.	The Blank OSL client receives the request.
3.		The Blank OSL client forwards the task to the LLM. The LLM generates the MCP message. The client sends the message directly to the appropriate MCP server for execution. The server executes the task and returns a result or error. The client forwards the final result (JSON data) or error (JSON-RPC error object) back to the Third-party Application.

3.6.2. Exceptional Scenario: Application Sends Incorrect Data

Step	Actor	System
1.	The application establishes a connection to the Blank OSL client's exposed MCP endpoint.	The Blank OSL client acts as an MCP host and accepts the connection.
2.	The application sends an MCP message that is non-compliant.	The client's API endpoint receives the malformed JSON-RPC message.
3.		The client's JSON-RPC parser rejects the message. The client returns a standard JSON-RPC error object to the Third-party Application. No task is sent to the LLM. No UI dialog is shown to the User.

3.6.3. Alternative Scenario: Third-Party Application Enters Prompt for the User

Step	Actor	System
1.	The application establishes a connection to the Blank OSL client's exposed MCP endpoint.	The Blank OSL client acts as an MCP host and accepts the connection.
2.	The application controls the user's mouse and keyboard and sends a task to the Blank OSL client.	The system takes the user's text and forwards it to the LLM for interpretation. The main terminal becomes non-interactive. The LLM interprets the task and generates a valid MCP message. The system displays the "Task Confirmation Dialog".